Major Project: Final Report

Authors:
Group 3: Zachary Lilley, Leslie Kelih, Javier Zertuche

1. Abstract

In this project we used the TurtleBot 4 robot platform by iCreate to solve the problem of finding an efficient way to conduct touring within an environment using predefined waypoints in a mapped area, while using a reactive facial recognition algorithm to confirm that it is being followed by tour participants. This simulated problem space reflects true challenges when it comes to creating a touring/guide robot, specifically for the SEC here on the University of Oklahoma Campus. In order to solve this problem we have implemented a D* Lite approach to search an occupancy grid that has been created using a map file and adding inflated layers using a dilation technique. Doing this, we found that for smaller spaces we could use a brute-force method to determine the shortest path between all nodes to solve the Traveling Salesperson Problem (TSP), however using D* Lite in this situation is sub-optimal because we are not taking advantage of its incremental replanning, however the facial recognition portion of the experiment proves useful for confirming following participants. Overall, we discovered that we can easily create a pseudo-topological search finding the best path among destinations, and further improvements can be made by passing continuous environment information, like LiDAR, to the planner to constantly determine the best path instead of planning once and executing.

2. Introduction

This project works off of the TurtleBot 4 base in order to model a touring interaction between a robot and humans. We aim to show that D* Lite and facial recognition can be used to support robotic navigation in dynamic environments and allow for better human-robot interactions. Using this platform we have access to several sensors and pre-written code, so we are able to focus on the main problems we want to solve rather than spending too much time working solely on boilerplate code. For the purposes of our experiment, we are concerned with creating a model of the occupancy grid of the navigation suite that is used and provided by ROS2 and the iCreate TurtleBot 4. This modeled grid is meant to represent what is available to the lower level systems of the robot when it uses those provided abstractions and implementations. Using our modeled grid, the D* Lite implementation will determine the best order of destinations (waypoints) to visit. This project is concerned with proving that D* Lite is capable of determining the best path to take to later be implemented as an actual planner for the navigation in order to take full advantage of D* Lite's incremental and dynamic planning algorithm. The scope of the project right now is to show that D* Lite can find the optimal path and solve the TSP, and allow interruption from a reactive face detector to confirm a follower is near. This is what is referred to as a Hybrid Robotic Control System [1]. The deliberative D* Lite planner is responsible for the highest-level instruction of the robot: given some points of interest, find the best possible route between them. The facial recognition interruptor is a reactive subsystem that, upon moving M meters, is triggered and waits to detect a face in the camera's image buffer before continuing. This hybrid control system and prototyping of the behavior of the robot will show that viability of D* Lite and facial recognition in the problem space of touring.

3. Approach & Methods

To begin, we have a set of fixed waypoints to represent your points of interest within a 2D map that will be navigated while the TurtleBot 4 periodically checks behind itself to look for a human face to confirm the tour participant is not lost. This approach uses ROS 2 Jazzy with the built in Nav2 orchestrator. The three main components that provide the behavior for this project are:

- The Planning Layer (tsp_executor): Decides waypoint order and dispatches goals.
- Execution Layer (Nav2 NavigateToPose action server): handles local control, recover, and replanning
- Perception Layer (face_detector): Published a /face_detected topic with a bool from the OAK-D camera stream

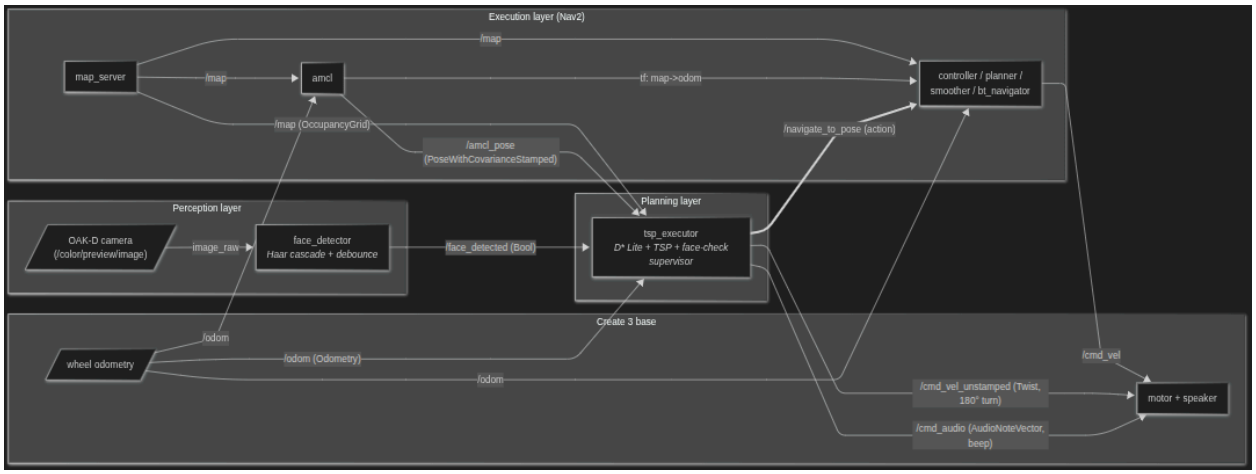

Figure 1: Block Diagram of ROS 2 Publisher/Subscriber Architecture

Next, we need to process the map in order to model what the Nav2 and Rviz2 use. To do this we use the topic /nav_msg/OccupanyGrid with a resolution of r (m/cell) which is 5 cm in our case, the origin is (o_x, o_y). This origin is used to conduct further calculations and processing on the map. We then build an obstacle mask where if M(i,j) is occupied it equals 1, and otherwise equals 0. Next we create an inflation layer for the planning algorithm to consider when planning, or else the robot will clip obstacles. This is done by calculating the ceiling of the ratio of the robots radius (18 cm) over the resolution of the map (5 cm), then using a dilation rule to where a cell becomes 1 if and only if any of its 8 neighbors or itself is 1, this is only done once per cell and is considered 0 by the rest of the dilation algorithm if it was flipped by the algorithm itself already. Finally, world-frame points (x, y) in meters are converted to grid indices by translating to the map origin (o_x, o_y) and scaling by the cell size r:

$$\text{col} = \lfloor (x - o_x)/r \rfloor, \qquad \text{row} = \lfloor (y - o_y)/r \rfloor$$

This culminates in a graph where the inflated occupancy grid is modeled as an 8-connected graph where cardinal transitions have an edge cost of 1, diagonal transitions cost √2, and any transition to an inflated cell costs infinity. For each of the 5 vertices (start + 4 waypoints) we run D* Lite

with that vertex as the goal in a fully-expanding mode that clears the priority queue, giving each cell's cost, g(s), a value. Then we can read the g value of each remaining vertex to fill a cost matrix such that the transition between each vertex can be easily looked up. The search uses the octile heuristic [2]:

$$h(a,b) = (\Delta x + \Delta y) + (\sqrt{2} - 2)\min(\Delta x, \Delta y)$$

Which is admissible and consistent on an 8-connected grid like ours, and is used together with D* Lite's standard priority (CalculateKey) key and rhs one step lookahead (which are potentially better informed than the g values themselves) [3]. One major caveat to using D* Lite as the planning algorithm, completely disjoint from the rest of the sensors and not being updated, is that compute_full refuses to terminate early and expansion order reduces down to Dijkstra's and the h(a,b) heuristic does not influence already visited cells. This full-fielded, static grid usage does not effectively use D* Lite's incremental strengths, and N runs on a plain Dijkstra would provide an identical cost matrix with less overhead per call. But we use the D* Lite implementation anyway because its benefits can be exploited in later experiments with dynamic obstacles and mid-journey replanning, where its incremental mechanisms would really pay off.

To select the tour, we simply keep track of the cheapest cost as we iterate through the cost matrix and calculate all possible paths that could be taken. We can eventually use something called the Held-Karp heuristic [4], but with only 5 nodes, it is cheap enough to simply brute-force the cheapest path. Once we add more than 7 nodes, Held-Karp outperforms brute force.

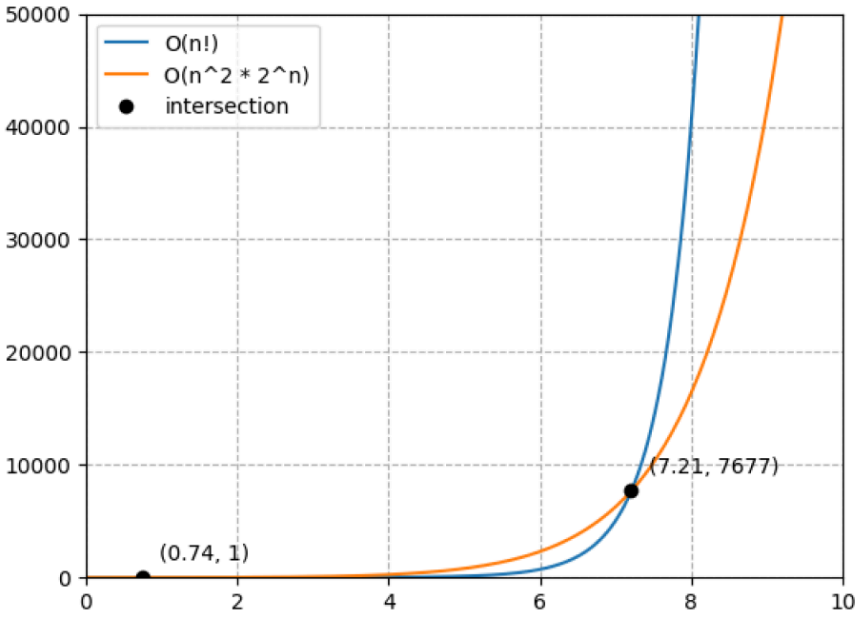

Figure 2: Shows Held-Karp outperforms brute-force after 7 nodes.

For executing our waypoint navigation and reactive interrupt behavior we do the following within the loops executed per-leg, these legs are the result of the identifying the path to navigate through the tsp_executor and D* Lite implementation.

a. Send a goal to NavigateToPose, where the goal is constructed in the context of the /map frame using the predefined goal points.
b. While the goal under the get_result_async() function is not done, we trigger the callbacks to accumulate an odometer until M meters is covered.
c. If the accumulated distance is greater than or equal to M, we cancel the Nav2 goal, run the face-check interrupt and wait for a true publication from the /face_detected topic.
d. Once a face is detected, the application resets the execution to that goal from its current position.

```
function navigate_to(wp):
  while ROS is running:
      reset distance_since_check to 0

      build Nav2 goal:
          frame = 'map'
          timestamp = now
          position = (wp.x, wp.y)
          orientation.w = 1.0  (heading doesn't matter)
      log "-> wp.name (x, y)"

      send goal to Nav2 asynchronously
      wait until Nav2 accepts or rejects the goal
      if goal was rejected or no handle returned:
          log error
          return False

      request async result future from the accepted goal
      interrupted = False

      while ROS running AND result not yet available:
          spin ROS once (timeout 100 ms) to process callbacks
          if distance_since_check >= face_check_distance_m:
              log "cancelling for face check"
              cancel the current nav goal (async) and wait for cancel to complete
              wait for result future to settle
              interrupted = True
              break

      if interrupted:
          run _face_check()  (turn-and-detect-face ritual)
          continue           (re-send same goal; Nav2 replans from new pose)

      status = result.status
      ok = (status == 4)   (STATUS_SUCCEEDED)
      log "wp.name finished status=... (OK/FAIL)"
      return ok

  return False
```

Figure 3: Pseudocode of Navigation Execution and Interrupt Behavior

Face check behavior itself triggers a set line of actions. First, we publish a manual Twist publication using a speed of 0.8 radians per second (rad/s) until the yaw integrated from the /odom topic, that is calculated using the standard quaternion to ZYX Euler approach, reaches

180° (~pi). We used the following formula to ensure that we wrap around at values higher than 2 pi:

$$\arctan 2(\sin(\theta_{\text{cur}} - \theta_{\text{last\_yaw}}),\ \cos(\theta_{\text{cur}} - \theta_{\text{last\_yaw}}))$$

The face check behavior will wait for 3 seconds before emitting a beeping sound from the robot. The beeps will continue every 3 seconds for 3 seconds until a face is detected and the pathing continues.

The face detection subsystem is implemented as a ROS 2 node called face_detector. This node subscribes to the OAK-D RGB preview stream on /oakd/rgb/preview/image_raw, converts each incoming sensor_msgs/Image message into an openCV BGR bridge, and then runs OpenCV's built-in Haar Cascade frontal-face detector on the frame. The image conversion supports multiple common image encodings such as bgr8, rgb8, mono8, bgra8, and rgba8, allowing the detector to handle different camera output formats without depending on cv_bridge.

Before detection, each frame is converted to grayscale and histogram equalization is applied to improve contrast under uneven lighting. The Haar Cascade detector then searches for frontal faces using configurable parameters such as scale_factor, min_neighbors, and min_face_size. These parameters allow the detector to be tuned depending on camera distance, lighting conditions, and how large the face is expected to appear in the OAK-D image stream.

To avoid the robot reacting to a single noisy frame, the node uses a short rolling detection window. Each frame contributes either a True value if at least one face is detected or a False value if no face is detected. The system only confirms a face when the number of positive detections inside the window reaches the configured min_hits threshold. In our configuration, this means that the detector can tolerate occasional missed frames, while still responding quickly when a person is actually visible.

Once the rolling-window is satisfied, the node publishes a std_msgs/Bool message on /face_detected. This topic is consumed by the tsp_executor during the reactive interrupt behavior. When

The tsp_executor manages the behavior loop of the robot. It calculates an optimal path of points to navigate to before sending those to a loop that will be subsumed once an odometry callback reaches a threshold of M meters. Once the threshold is met, the robot turns 180° and scans for a face, if a face is not detected after three seconds, the robot beeps for one second every three seconds to help tour participants find the robot. Once a face is detected, the robot continues the tour until the threshold of distance M meters is met. This continues until the robot finishes the tour.

To reproduce this experiment you will need:

- A TurtleBot 4 (iRobot Create 3 + Raspberry Pi 4 + OAK-D-Lite), running with ROS 2 Jazzy, Nav2, and Rviz2.
- AMCL localization seeded by RViz 2D Pose Estimate.

- A previously generated occupancy grid map with a resolution of 5 cm.
- 4 fixed (x, y) coordinates generated from the grid map.

4. Results

So far we are still integrating the facial recognition of the experiment. But we have found the D* Lite static search to be effective and consistent at planning the best order of goals to execute a tour. Some instances of network instability causes LiDAR to publish inconsistently, causing Nav2 dynamic pathing to struggle which path to follow given the goal orders. We mostly resolved this by removing dead processes left over from testing with other TurtleBot 4 robots. We found that the average planning period for the D* Lite brute force is about 5 seconds on average. The facial recognition implementation of the application works well quickly detecting a face during the reactive behavior, but due to the use of Haar Cascade, which we use because of its pre-deep-neural-network nature and low computing cost, the algorithm is biased against those faces that are different than the data it was created on. We could use OpenCV's Deep Neural Network [7] which works well with limited computing hardware and outperforms Haar Cascade on every front. There are some issues right now with LiDAR artifacting due to networking and hardware issues caused by the amount of data being published across the delivery network. This causes the internal Nav2 navigation function to overreact to these perceived obstacles. To fix this we need to narrow in on the configurations for the LiDAR within the nav2_config.yaml file. Overall the project was a success because we were able to model the metric map that the Nav2 sees to correctly estimate an optimal path with D* Lite, and have the path executor subsume the navigation loop by creating an odometry callback that triggers once a certain distance has been covered, then releases once the face topic publishes a hit.

5. Discussion

One of the glaring issues of this project is the use of D* Lite to solve the travelling salesman problem. As mentioned briefly above, the main benefit of using D* Lite is its reverse planning algorithm and incremental path repairing. It starts at the goal and expands backwards, only repairing paths that are interrupted and not replanning the path entirely, as one would have to in A* search. The problem is that not only are we not updating the path once the cheapest order of goals is found, D* Lite is concerned with finding the shortest path between two endpoints. It has no notion of "visit these N waypoints in the best order. For instance, if you feed D* Lite a sequence of points and just let it plan leg-by-leg in the order you hand them in, you are simply going to find the best path for each of those legs, not the best path between them all. This is why we had to brute force and manually check all N! possible paths, and find the total path with the cheapest cost:

```
n = number of nodes

# Build n x n cost matrix; cost[i][j] = planned path cost from node i to node j
create cost matrix of size n x n, filled with INF

for each node j in 0..n-1:
    planner = DStarLite(obstacles, goal = cells[j])
    planner.compute_full() # expand the entire grid once, not just to a single start

    for each node i in 0..n-1:
        if i == j:
            cost[i][j] = 0
        else:
            cost[i][j] = planner.cost_to_goal(cells[i])
    log "cost field computed with goal=nodes[j].name"

# Brute-force TSP (open path): start fixed at index 0, permute the rest
wp_indices = [1, 2, ..., n-1]
best_perm = None
best_cost = INF

for each perm in permutations(wp_indices):    # O((n-1)!)
    total = 0
    prev = 0                                  # start node
    for idx in perm:
        total += cost[prev][idx]
        prev = idx
    if total < best_cost:
        best_cost = total
        best_perm = perm
```

Figure 4: D* Lite TSP Brute-force Cost Method

In order to take advantage of the incremental repairing features of D* Lite, we should utilize a combination of A* to get the true path cost c(i, j). This would build an NxN cost matrix. For a real situation in the SEC where there are up to 10-20 tour locations, this would be totally traceable with about 100-400 A* calls, with most of them being short. We could also implement Held-Karp to shortcut the algorithm as well to further increase performance of high-level path planning. The real practical use of D* Lite would come from using it during drastic environment changes, such as broken elevators, extremely crowded corridors, or something like furniture blocking a route. We could use a dynamic TSP implementation utilizing D* Lite to continuously identify a current, most up-to-date. high-level pathing plan [6]. There is some literature on this, but it seems largely over-complicated for the implementation of a tour robot.

The way that Nav2 actually calculates a path based on a map, either created by SLAM on the fly or previously generated, eventually the NavigateToPose function behavior tree hits a ComputePathToPose action, which calls into the nav2_planner server. This serve is a plugin host

that loads whichever global planner is configured, the default planner is Smac 2D, an A* 2D-grid search that is aware of the footprint of the robot against the costmap at each candidate pose. While Smac is good for static, slowly changing environment, Smac is a one-shot search, so every time the behavior tree's RateController fires the planner discards its prior work and re-runs A* from scratch against the updated cost map. In highly dynamic environments, such as that of the SEC, this repeated full-graph search is wasteful since the majority of cells have not changed between invocations, yet their values are recomputed anyway. D* Lite addresses this directly. As an incremental heuristic search, it maintains a consistent search tree across replans and when cost maps' cells change only the nodes whose values have become inconsistent with their neighbors are re-calculated. This results in the same optimal path that A* would produce, but at a much cheaper cost per replanning. This is where the value of D* Lite outshines Smac 2D's from-scratch planning approach. This is the main reason we wanted to show a proof-of-concept using D* Lite, despite its current exclusion from the global planner and isolation from cost map updates.

6. Conclusions

Overall, the project has shown that it is possible to use D* Lite to solve the travelling salesman problem. We have shown that it is possible to interrupt the optimal high-level planning algorithm with a reactive subsumption to conduct facial recognition without losing the high-level plan. Despite its current isolated state, we have shown the ability to use D* Lite on the ROS 2 platform by decomposing the problem into two layers: a TSP solver that handles a cost matrix in the form of a brute-forced iterator and best-cost tracker, and a D* Lite cost calculator that actually determines the cost of each leg in the matrix. However, it is glaringingly obvious that we have barely scratched the surface of solving this problem of dynamic path finding. There are still several items that are weak in this project such as the isolation of the D* Lite algorithm from the real cost map and it being used solely as a cost calculator for a high-level plan. The face detection implementation is lightweight and highly portable as it uses a /face_detected topic to send updates, and is highly desirable for several applications for the TurtleBot 4. In conclusion, the project set out to show how D* Lite could be used to make high-level plans and allow interrupt behavior and it has done that, several improvements can be made, however the initial work here allows room for improvement and further iterations easily.

7. Future Work

In the future, the highest priorities are implementing the D* Lite search on the nav2_planner server, this is more sophisticated as it has to be done in C++, and conform to the GlobalPlanner plugin interface, which requires proper lifecycle management, parameter handling through the ROS 2 parameter server, and safe parallel processing access to the shared cost map rather than the standalone prototype we have been developing against synthetic grids. The plugin will also need to expose its incremental state correctly across future computePath calls since the default planner invocation pattern just assumes a stateless planner and we will need to implement a way for D* Lite cost matrix to persist between searches. Once the planner is available, we will need to find a way to allow interrupts for subsumption behavior, namely the face detector behavior The

behavior should be extended to be able to subsume the current navigation goal when a visitor is detected and the robot should pause to engage with the person rather than the simple current experiment where it is simply confirming it is being followed. In the Nav2 framework this is most likely going to be implemented at the behavior tree rather than inside the planner itself: a custom BT condition node monitoring the face detector topic could trigger a ReactiveFallback that suspends the FollowPath execution that then allows the engagement to take place and resumes the tour when the interaction ends. This could then be extended to work with more complex facial recognition instead of Haar Cascade, which has been shown to be biased towards certain faces. Beyond these two priorities, additional work for the future includes using QR codes to conduct local control strategies at each waypoint to better position the robot at goals, conducting empirical benchmarking of D* Lite replan times against Smac 2D, and extending the TSP calculation to incorporate a type of timeouts if the tour is ever constrained to a fixed duration.

8. Bibliography

[1] J. S. Albus, "A Reference Model Architecture for Intelligent Systems Design," NIST Interagency/Internal Report NISTIR 5502, National Institute of Standards and Technology, Gaithersburg, MD, 1993.

[2] A. Patel, “Heuristics,” *theory.stanford.edu*, 1997. https://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html

[3] Koenig and M. Likhachev, "D* Lite," in *Proc. AAAI Conf. Artificial Intelligence (AAAI-02)*, Edmonton, AB, Canada, 2002, pp. 476–483.

[4] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *Journal of the Society for Industrial and Applied Mathematics*, vol. 10, no. 1, pp. 196–210, Mar. 1962.

[5] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Kauai, HI, USA, 2001, pp. 511–518.

[6] V. Pillac, M. Gendreau, C. Guéret, and A. L. Medaglia, "A review of dynamic vehicle routing problems," *Eur. J. Oper. Res.*, vol. 225, no. 1, pp. 1–11, Feb. 2013.

[7] OpenCV, "Deep Neural Networks (dnn module)," OpenCV Documentation, 2025. [Online]. Available: https://docs.opencv.org/4.x/d2/d58/tutorial_table_of_content_dnn.html. [Accessed: May 1, 2026

9. Appendices

a. Full source code

i. github.com/kolchefen/ItIR_FP

b. Launch File Description

i. The bringup.launch.py file is the entry point to the navigation module and manages the startup of localization, navigation, and visualization. All nodes are grouped into one process called nav2_container for optimal performance. The map server and AMCL node are placed in the same memory space, so that communication between the nodes cannot be hindered by any network issues. This keeps lifecycle bonds in process and avoids the DDS churn that caused AMCL bond timeouts when localization ran in separate processes. This is relevant because strong lifecycle connections between the stack modules are needed for stability.

c. Launch and Operation Instructions

i. Prerequisites

1. ROS 2 workspace rooted at ~/proj
2. Have the code be in ~/proj/ItIR_FP
3. Place map file at ~/proj/area_map/map_area.yaml
4. Every terminal must source the workspace overlay with source install/setup.bash
5. Must have waypoints defined in the waypoints.py that are relevant to the map:

```
WAYPOINTS: tuple[Waypoint, ...] = (
    Waypoint(name="A", x=-1.4907296895980835, y=1.1794612407684326),
    Waypoint(name="B", x=-3.8711657524108887, y=0.8193239569664001),
    Waypoint(name="C", x=-0.19871702790260315, y=-0.7073472142219543),
    Waypoint(name="D", x=0.53733891248703, y=0.7723487019538879),
)
```

ii. Step 1: Build the Package

1. From ~/proj, run:
   `colcon build --packages-select turtlebot4_reactive_controller`
2. Source the overlay: `source install/setup.bash`

iii. Step 2: Run the robot publishers

1. ssh into the OU tb4 and run:
   `ros2 launch turtlebot4_bringup standard.launch.py`

iv. Step 3: Launch the bringup

1. For each additional terminal you must conduct the robot-setup.sh as described in the OU_turtlebots4.pdf, and you must source the built package with `source install/setup.bash`
2. In a sourced terminal Run:
   `ros2 launch turtlebot4_reactive_controller bringup.launch.py map:=~/proj/area_map/map_area.yaml`

v. Step 4: TSP Executor

1. In another sourced terminal run:
   ```
   ros2 run turtlebot4_reactive_controller tsp_executor
   ```

vi. Step 5: Face detector
   1. In another sourced terminal run:
      ```
      ros2 run turtlebot4_reactive_controller face_detector
      ```

vii. Step 5: Localize the robot
   1. Click **2D Pose Estimate** in RViz and click-drag where the robot actually is on the map
   2. The AMCL module will not publish on /amcl_pose until this is done, and the TSP executor blocks waiting on that topic.

viii. Step 6: When the robot beeps, present a face to the camera to continue the tour.

d. Contributions

i. Zachary Lilley worked on the D* Lite implementation, Launch File, and TSP Executor.

ii. Javier Zertuche Worked on facial recognition and its topic publication and acceptance.

iii. Leslie Kelih debugged, tested, and reviewed the experiment and solved networking issues.

iv. Group 9 provided us with code in kill-useless.txt that removes previous ros-domain processes that are left over when you switch to different robots.

v. Group 2 provided help with implementing beep functionality in tsp_executor.py:198-208

vi. Groups 1, 6, and 7 used our map_area.pgm and map_area.yaml, as well as our cardboard city environment.